

Voici un **plan de mise en œuvre complet et professionnel** pour **HyperRuleEx**, rédigé de manière claire et structurée, avec une justification des choix techniques.

Plan de Mise en Œuvre : HyperRuleEx

Un modèle hybride unifié combinant règles logiques explicites et contre-factuels diversifiés pour données tabulaires

1. Objectif du modèle

Concevoir un **seul réseau de neurones** capable de délivrer, en un seul forward pass (ou quelques passes contrôlées) :

- Une prédiction fiable
 - Des **règles IF-THEN** interprétables (inspiré de HyperLogic)
 - Des **contre-factuels diversifiés** mais plausibles pour chaque classe alternative (amélioration de HyConEx)
 - Des **contributions des features** (feature importance) interprétables même sur les données originales non binarisées
-

2. Architecture générale de HyperRuleEx

Entrée :

- x : vecteur de features **binarisées** (+1 / -1)
- ε : vecteur de bruit gaussien $\sim \mathcal{N}(0, \sigma^2)$ de même dimension que x

Hypernetwork (cœur du modèle) :

- Architecture : TabResNet (ou FT-Transformer)
- Entrée : concaténation $[x ; \varepsilon]$ (dimension 2D)
- Sorties simultanées :
 - $(w, u) \rightarrow$ pour les règles logiques (HyperLogic)
 - $V \in \mathbb{R}^{\{K \times D\}} \rightarrow$ directions pour les contre-factuels (HyConEx)

Deux branches parallèles :

Branche 1 – HyperLogic (Règles & Prédiction)

- Rule Layer : calcul des activations logiques o_k
- OR Layer : combinaison pondérée $f(x) = \sum u_k o_k$
- Prédiction finale : $\hat{y} = \text{sigmoid}(f(x))$ ou argmax

Branche 2 – Counterfactuals

- Génération : $x'_m = x - \alpha_m \cdot V_m$ pour chaque classe $m \neq \hat{y}$
- Vérification de plausibilité via un **Normalizing Flow conditionnel** (MAF)

3. Justification des choix techniques

Choix Technique	Justification
Hypernetwork comme générateur	Permet de générer à la fois les poids des règles et les directions de contre-factuels de manière conditionnée par l'entrée
Concaténation $[x ; \varepsilon]$	Permet de contrôler la diversité tout en gardant une forte dépendance à l'exemple x . Meilleur compromis que l'addition
Bruit gaussien sur ε	Simple, différentiable, et bien maîtrisé dans la littérature des hypernetworks
Maintien de la structure de HyperLogic	Garde l'excellente interprétabilité des règles logiques (IF-THEN)
Utilisation de $V \in \mathbb{R}^{K \times D}$	Permet d'avoir une direction dédiée par classe cible (comme dans HyConEx)
Normalizing Flow (MAF)	Meilleure méthode actuelle pour estimer la densité et garantir la plausibilité des contre-factuels sur données tabulaires
Loss de stabilité (Consistency Loss)	Résout le conflit entre diversité et stabilité de la prédiction

4. Sortie du modèle et calcul des explications

A. Prédiction finale

Le modèle prédit **uniquement** à partir de la branche HyperLogic :

$$f(x) = \sum_{k=1}^K u_k \cdot h \left(\sum_{d=1}^D w_{d,k} x_d - \sum_{d=1}^D |w_{d,k}| \right)$$

$$\hat{y} = \arg \max_k f(x) \quad \text{ou} \quad \text{sigmoid}(f(x))$$

B. Règles IF-THEN

À partir de (w, u) :

- Pour chaque règle k où $|u_k| > \text{seuil}$:
 - Conditions positives : features où $w_{\{d,k\}} > 0.5 \rightarrow \text{feature}_d = \text{True}$
 - Conditions négatives : features où $w_{\{d,k\}} < -0.5 \rightarrow \text{feature}_d = \text{False}$
- Contribution de la règle = u_k

C. Contre-factuels

Pour chaque classe cible $m \neq \hat{y}$:

$$x'_m = x - \alpha_m \cdot V_m$$

α_m est choisi par une petite recherche linéaire pour maximiser la probabilité de la classe m tout en minimisant la distance.

D. Contributions des features (Feature Importance)

Deux niveaux sont possibles :

1. Niveau local (par règle) :

Contribution d'une feature = somme des $|w_{\{d,k\}}| \times |u_k|$ pour toutes les règles actives.

2. Niveau global (par feature) :

Moyenne sur tout le dataset des contributions locales.

5. Retour des features de l'état binarisé vers l'état original (non-binarisé)

C'est une étape cruciale pour rendre le modèle utilisable en pratique.

C'est une étape **cruciale** pour rendre le modèle utilisable en pratique.

Méthode proposée :

On maintient un **mapping de correspondance** pendant le prétraitement :

- Pour chaque feature originale (ex: `age`), on garde en mémoire :
 - Les seuils de discrétisation utilisés
 - Les noms des prédicats binaires créés (ex: `age_[30-50]`, `age_[50+]`)

Lors de l'interprétation :

- Pour chaque règle ou contre-factuel, on **regroupe** les prédicats binaires qui correspondent à la même feature originale.
- On traduit les conditions binaires en langage naturel compréhensible :
 - Exemple : $\text{age}_{[30-50]} = \text{True} \wedge \text{age}_{[50+]} = \text{False} \rightarrow 30 \leq \text{age} \leq 50$
- Pour les contre-factuels : on applique l'inverse de la binarisation pour revenir aux valeurs réelles (ex: changer `age` de 42 à 61).

Implémentation :

- Créer un dictionnaire `binary_to_original` pendant le prétraitement
 - Créer une fonction `interpret_rule(rule, mapping)` et `interpret_counterfactual(x_cf, mapping)`
-

6. Plan de mise en œuvre résumé

Phase 1 : Préparation des données

- Binarisation + mapping de correspondance

Phase 2 : Architecture

- Hypernetwork avec entrée $[x; \epsilon]$
- Deux sorties : (w, u) et v

Phase 3 : Fonction de perte

- $L_{\text{HyperLogic}}$ (BCE + diversité + sparsité)

- L_{conEx} (validité + proximité + plausibilité)
- $L_{stability}$ (KL ou MSE entre prédiction propre et bruitée)

Phase 4 : Inférence

- Prédiction via HyperLogic
 - Génération de plusieurs contre-factuels via échantillonnage de ϵ
 - Traduction des règles et contre-factuels vers l'espace original
-

Veux-tu que je développe maintenant une section en particulier avec plus de détails (perte totale, pseudo-code, ou schéma) ?

Ou préfères-tu que je te rédige directement le document complet (style rapport technique) ?